Lab 8 – Programs on Matrix using CUDA

Q1. Write a program in CUDA to add two matrices for the following specifications:
a. Each row of resultant matrix to be computed by one thread.
b. Each column of resultant matrix to be computed by one thread.
c. Each element of resultant matrix to be computed by one thread.

Codes:

Q1a.cu

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__global__ void addRows(int *A, int *B, int *C, int N) {
int row = blockIdx.x * blockDim.x + threadIdx.x;
if (row < N) {
for (int col = 0; col < N; col++) {
int idx = row * N + col;
C[idx] = A[idx] + B[idx];
}
}
}

int main() {
int N;
printf("Enter size N for NxN matrix addition: ");
scanf("%d", &N);

size_t size = N * N * sizeof(int);
int *h_A = (int*)malloc(size);
int *h_B = (int*)malloc(size);
int *h_C = (int*)malloc(size);

printf("Enter %d elements for Matrix A:\n", N * N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_A[i]);

printf("Enter %d elements for Matrix B:\n", N * N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_B[i]);

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
```

```
cudaMalloc(&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

printf("Launching %d threads (one per row).\n", N);
addRows<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix C (Row-wise Addition):\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_C[i * N + j]);
}
printf("\n");
}
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);

return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc q1a.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter size N for NxN matrix addition: 3
Enter 9 elements for Matrix A:
1 2 3
4 5 6
7 8 9
Enter 9 elements for Matrix B:
1 2 3
4 5 6
7 8 9
Launching 3 threads (one per row).

Resultant Matrix C (Row-wise Addition):
2       4       6
8       10      12
14      16      18
```

Q1b.cu

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__global__ void addCols(int *A, int *B, int *C, int N) {
int col = blockIdx.x * blockDim.x + threadIdx.x;
if (col < N) {
for (int row = 0; row < N; row++) {
int idx = row * N + col;
C[idx] = A[idx] + B[idx];
}
}
}

int main() {
int N;
printf("Enter size N for NxN matrix addition: ");
scanf("%d", &N);

size_t size = N * N * sizeof(int);
int *h_A = (int*)malloc(size);
int *h_B = (int*)malloc(size);
```

```
int *h_C = (int*)malloc(size);

printf("Enter %d elements for Matrix A:\n", N * N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_A[i]);

printf("Enter %d elements for Matrix B:\n", N * N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_B[i]);

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

printf("Launching %d threads (one per column).\n", N);
addCols<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix C (Column-wise Addition):\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_C[i * N + j]);
}
printf("\n");
}

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc q1b.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter size N for NxN matrix addition: 3
Enter 9 elements for Matrix A:
1 1 1
2 2 2
3 3 3
Enter 9 elements for Matrix B:
1 1 1
2 2 2
3 3 3
Launching 3 threads (one per column).

Resultant Matrix C (Column-wise Addition):
2       2       2
4       4       4
6       6       6
```

Q1c.cu

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__global__ void addElements(int *A, int *B, int *C, int N) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
if (row < N && col < N) {
int idx = row * N + col;
C[idx] = A[idx] + B[idx];
}
}

int main() {
int N;
printf("Enter size N for NxN matrix addition: ");
scanf("%d", &N);

size_t size = N * N * sizeof(int);
int *h_A = (int*)malloc(size);
int *h_B = (int*)malloc(size);
int *h_C = (int*)malloc(size);
```

```
printf("Enter elements for Matrix A:\n");
for (int i = 0; i < N * N; i++) scanf("%d", &h_A[i]);

printf("Enter elements for Matrix B:\n");
for (int i = 0; i < N * N; i++) scanf("%d", &h_B[i]);

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (N + 15) / 16);

printf("Launching N*N threads (one per element).\n");
addElements<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix C (Element-wise Addition):\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_C[i * N + j]);
}
printf("\n");
}

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc q1c.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter size N for NxN matrix addition: 3
Enter elements for Matrix A:
1 1 1
1 1 1
1 1 1
Enter elements for Matrix B:
2 2 2
2 2 2
2 2 2
Launching N*N threads (one per element).

Resultant Matrix C (Element-wise Addition):
3       3       3
3       3       3
3       3       3
```

Q2. Write a program in CUDA to multiply two matrices for the following specifications:
a. Each row of resultant matrix to be computed by one thread.
b. Each column of resultant matrix to be computed by one thread.
c. Each element of resultant matrix to be computed by one thread.

Codes:

Q2a.cu

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__global__ void matMulRowWise(int *A, int *B, int *C, int N) {
int row = blockIdx.x * blockDim.x + threadIdx.x;
if (row < N) {
for (int col = 0; col < N; col++) {
int sum = 0;
for (int k = 0; k < N; k++) {
sum += A[row * N + k] * B[k * N + col];
}
C[row * N + col] = sum;
}
}
```

```
}

int main() {
int N;
printf("Enter the size N for NxN matrices: ");
scanf("%d", &N);
size_t size = N * N * sizeof(int);

int *h_A = (int*)malloc(size);
int *h_B = (int*)malloc(size);
int *h_C = (int*)malloc(size);

printf("Enter elements for Matrix A (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_A[i]);

printf("Enter elements for Matrix B (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_B[i]);

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

printf("Launching %d threads to handle %d rows.\n", N, N);
matMulRowWise<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix C (Row-wise computation):\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_C[i * N + j]);
}
printf("\n");
}

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);
```

```
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc q2a.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter the size N for NxN matrices: 3
Enter elements for Matrix A (3x3):
1 1 1
1 1 1
1 1 1
Enter elements for Matrix B (3x3):
1 1 1
1 1 1
1 1 1
Launching 3 threads to handle 3 rows.

Resultant Matrix C (Row-wise computation):
3       3       3
3       3       3
3       3       3
```

Q2b.cu

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__global__ void matMulColWise(int *A, int *B, int *C, int N) {
int col = blockIdx.x * blockDim.x + threadIdx.x;
if (col < N) {
for (int row = 0; row < N; row++) {
int sum = 0;
for (int k = 0; k < N; k++) {
sum += A[row * N + k] * B[k * N + col];
}
C[row * N + col] = sum;
}
}
}

int main() {
int N;
```

```
printf("Enter the size N for NxN matrices: ");
scanf("%d", &N);
size_t size = N * N * sizeof(int);

int *h_A = (int*)malloc(size);
int *h_B = (int*)malloc(size);
int *h_C = (int*)malloc(size);

printf("Enter elements for Matrix A (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_A[i]);

printf("Enter elements for Matrix B (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_B[i]);

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

printf("Launching threads to handle %d columns.\n", N);
matMulColWise<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix C (Column-wise computation):\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_C[i * N + j]);
}
printf("\n");
}

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc q2b.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter the size N for NxN matrices: 3
Enter elements for Matrix A (3x3):
 2 2 2
 2 2 2
 2 2 2
Enter elements for Matrix B (3x3):
 2 2 2
 2 2 2
 2 2 2
Launching threads to handle 3 columns.

Resultant Matrix C (Column-wise computation):
12      12      12
12      12      12
12      12      12
```

Q2c.cu

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__global__ void matMulElementWise(int *A, int *B, int *C, int N) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
if (row < N && col < N) {
int sum = 0;
for (int k = 0; k < N; k++) {
sum += A[row * N + k] * B[k * N + col];
}
C[row * N + col] = sum;
}
}

int main() {
int N;
printf("Enter the size N for NxN matrices: ");
scanf("%d", &N);

size_t size = N * N * sizeof(int);
```

```
int *h_A = (int*)malloc(size);
int *h_B = (int*)malloc(size);
int *h_C = (int*)malloc(size);

printf("Enter elements for Matrix A (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_A[i]);

printf("Enter elements for Matrix B (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) scanf("%d", &h_B[i]);

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (N + 15) / 16);
printf("Launching a 2D grid of threads.\n");

matMulElementWise<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix C (Element-wise computation):\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_C[i * N + j]);
}
printf("\n");
}

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);

return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc q2c.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter the size N for NxN matrices: 3
Enter elements for Matrix A (3x3):
3 3 3
3 3 3
3 3 3
Enter elements for Matrix B (3x3):
3 3 3
3 3 3
3 3 3
Launching a 2D grid of threads.

Resultant Matrix C (Element-wise computation):
27      27      27
27      27      27
27      27      27
```

A1. Write a CUDA program that reads a MXN matrix A and produces a resultant matrix B of same size as follows: Replace all the even numbered matrix elements with their row sum and odd numbered matrix elements with their column sum.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__global__ void replaceEvenOdd(int *A, int *B, int M, int N) {
int r = blockIdx.y * blockDim.y + threadIdx.y;
int c = blockIdx.x * blockDim.x + threadIdx.x;
if (r < M && c < N) {
int val = A[r * N + c];
int sum = 0;

if (val % 2 == 0) {
for (int j = 0; j < N; j++) {
sum += A[r * N + j];
}
} else {
for (int i = 0; i < M; i++) {
sum += A[i * N + c];
}
```

```
}
B[r * N + c] = sum;
}
}

int main() {
int M, N;
printf("Enter number of rows (M) and columns (N): ");
scanf("%d %d", &M, &N);

size_t size = M * N * sizeof(int);
int *h_A = (int*)malloc(size);
int *h_B = (int*)malloc(size);

printf("Enter the %d elements of the matrix:\n", M * N);
for (int i = 0; i < M * N; i++) {
scanf("%d", &h_A[i]);
}

int *d_A, *d_B;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (M + 15) / 16);

replaceEvenOdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, M, N);
cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix B:\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_B[i * N + j]);
}
printf("\n");
}

cudaFree(d_A); cudaFree(d_B);
free(h_A); free(h_B);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc a1.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter number of rows (M) and columns (N): 2 3
Enter the 6 elements of the matrix:
1 2 3
4 5 6

Resultant Matrix B:
5       6       9
15      7       15
```

A2. Write a CUDA program to read a matrix A of size NXN. It replaces the principle diagonal elements with zero. Elements above the principle diagonal by their factorial and elements below the principle diagonal by their sum of digits.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include "device_launch_parameters.h"
#include <cuda_runtime.h>

__device__ int getFactorial(int n) {
if (n < 0) return 0;
if (n == 0) return 1;
int res = 1;
for (int i = 1; i <= n; i++) {
res *= i;
}
return res;
}
__device__ int getSumDigits(int n) {
int sum = 0;
n = (n < 0) ? -n : n;
while (n > 0) {
sum += n % 10;
n /= 10;
}
return sum;
}
__global__ void processMatrix(int *A, int N) {
int r = blockIdx.y * blockDim.y + threadIdx.y;
int c = blockIdx.x * blockDim.x + threadIdx.x;
if (r < N && c < N) {
int idx = r * N + c;
```

```
if (r == c) {
A[idx] = 0;
}
else if (r < c) {
A[idx] = getFactorial(A[idx]);
}
else {
A[idx] = getSumDigits(A[idx]);
}
}
}

int main() {
int N;
printf("Enter the size of the NXN matrix: ");
size_t size = N * N * sizeof(int);
int *h_A = (int*)malloc(size);

printf("Enter the %d elements of the matrix:\n", N * N);
for (int i = 0; i < N * N; i++) {
scanf("%d", &h_A[i]);
}
int *d_A;
cudaMalloc(&d_A, size);
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((N + 15) / 16, (N + 15) / 16);

processMatrix<<<blocksPerGrid, threadsPerBlock>>>(d_A, N);
cudaMemcpy(h_A, d_A, size, cudaMemcpyDeviceToHost);

printf("\nResultant Matrix:\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%d\t", h_A[i * N + j]);
}
printf("\n");
}

cudaFree(d_A);
free(h_A);
return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L8$ nvcc -o abc a2.cu
6CSEC1@debian:~/Documents/230905152_PPL/L8$ ./abc
Enter the size of the NXN matrix: 3
Enter the 9 elements of the matrix:
1 2 3
4 5 6
7 8 9

Resultant Matrix:
0       2       6
4       0       720
7       8       0
```